État d'implémentation face à la gouvernance

> ***Statut : constat.*** *Ce document décrit ce qui a été vérifié dans le code, avec ses sources. Les objectifs et les plans sont dans `cible/`.*

> *Responsable : Cédric · Version 0.1 du 20/10/2026 · Relevé sur `main` après la fusion des PR #14 (code de la plateforme dans `app/`), #16 (WAF) et #17 (SIEM)*
>
> *Les documents de gouvernance ont été écrits avant le code. Ce registre dit, pour chaque hypothèse importante, si le code la **confirme**, la **contredit** ou ne l'a **pas encore implémentée**. Il est à mettre à jour après chaque fusion importante et avant les essais (WOR-73).*

1. Ce qui existe au 20/10/2026

Composant	État	Où
API de paiement (Node.js) avec authentification par session, idempotence des paiements, métriques Prometheus, journaux JSON	Implémentée	`app/backend/`
Front web et page d'administration	Implémentés	`app/front/`
Base PostgreSQL de l'application	Implémentée, **sans sauvegarde**	`app/infra/docker-compose.yml`, `app/infra/database/init/001_schema.sql`
Socle : Vault, PostgreSQL, Prometheus (collecte toutes les 5 s), Loki (rétention 7 jours), Grafana	Implémenté, **mais l'API n'y est pas encore branchée**	`infra/docker-compose.yml`
Extraction des features de détection (fenêtre de 60 s, pas de 10 s)	Implémentée	`ml/features/extract.py`
Modèle de détection, envoi des alertes	**Absent**	`ml/training/`, `ml/inference/` vides
Moteur de remédiation	**Absent**	`services/remediation/README.md` : « À implémenter »
WAF ModSecurity + OWASP CRS, en mode bloquant, seul point d'entrée publié du socle	Implémenté. **Mais l'API n'est pas encore derrière lui** : le trafic légitime reçoit une erreur 502, les injections sont bloquées en 403. Pas d'interface pour ajouter un blocage à chaud : les règles changent par fichier, puis redémarrage	`infra/docker-compose.yml`, `infra/waf/README.md`

Composant	État	Où
Tableau de bord SOC / SIEM dans Grafana : blocages du WAF, accès à Vault, flux corrélé WAF · Vault · PostgreSQL	Implémenté ; **aucune règle d'alerte**	`infra/grafana/provisioning/dashboards/soc-siem.json`
Tableau de bord d'incident (WOR-40)	**Absent** ; la page d'administration affiche des statuts fixes	`services/dashboard/`, `app/backend/api/src/main.ts`
Remise à l'état sain	Implémentée ; **efface base, journaux, métriques et Vault**	`scripts/reset-known-good.sh`
Assistant RAG en lecture seule	Implémenté	`rag/docker-compose.rag.yml`
Scripts d'attaque A et B	Implémentés, **à recâbler sur la vraie API** (voir §3)	`attack/`

Il existe **quatre déploiements Docker Compose séparés** (application, socle, labo IA, RAG), avec deux bases PostgreSQL différentes. Tant qu'ils ne sont pas réunis, la chaîne attaque → détection → remédiation → vérification ne peut pas être jouée de bout en bout.

2. Hypothèses de la gouvernance confrontées au code

Hypothèse	Document	Verdict	Constat
Aucune transaction confirmée perdue (RPO = 0)	`cible/rto-rpo.md`	**Non implémenté**	Aucune sauvegarde, aucun archivage des journaux de la base. La seule « reprise » des données est la remise à l'état sain, qui repart des données de départ
Après une injection : snapshot puis rejeu des transactions légitimes	`cible/rto-rpo.md` §5, `cible/pca-pra.md` S2	**Non implémenté**	Pas de snapshot. Point toujours à trancher avec Michael et Reza
Deux signaux concordants avant une action destructrice	`cible/pca-pra.md` S4	**Non implémenté**	Ni moteur, ni modèle. Les runbooks parlent de deux fenêtres consécutives d'une même famille, ce qui n'est pas deux signaux indépendants
Au plus 3 remédiations en 10 minutes par service	`cible/pca-pra.md` §3	**Non implémenté**	Les runbooks prévoient une escalade après 2 tentatives (`docs/runbooks/reprise.md`) : à harmoniser
Journaux copiés avant toute restauration	`cible/pca-pra.md` S2, §6	**Contredit**	La remise à l'état sain supprime les journaux Loki sans copie
Assistant RAG sans accès privilégié	`cible/pca-pra.md` §5	**Confirmé**	Conteneur en lecture seule, sans droits ni accès au moteur Docker
Hôte unique, aucune sauvegarde hors de l'hôte	`cible/nis2-dora.md`, `cible/pca-pra.md` S7	**Confirmé**	Inventaire Ansible local uniquement

Hypothèse	Document	Verdict	Constat
Secrets dans HashiCorp Vault	`cible/ebios-rm.md` BS9	**Partiellement contredit**	Vrai pour le socle. L'application a son mot de passe de base en clair dans son fichier Compose, et sa clé d'API OpenAI en variable d'environnement
Collecte toutes les 5 s, détection en 20 à 70 s	`cible/rto-rpo.md` §4	**En partie confirmé**	Collecte à 5 s et fenêtre de 60 s confirmées. Le délai de détection ne peut pas être mesuré : il n'y a pas encore de détecteur
Une injection altère des données, qu'il faut ensuite restaurer	`cible/pca-pra.md` S2	**Contredit par le WAF**	Le WAF du socle bloque 9 vecteurs sur 10 de l'attaque B en 403, avant l'API (`infra/waf/README.md`). La protection est démontrée ; la restauration ne peut pas l'être tant qu'aucune injection n'atteint la base
Vérification de la reprise par une transaction réelle	`cible/rto-rpo.md` §1	**À construire**	`/health` répond toujours « ok » sans rien vérifier : il ne peut pas servir de preuve de reprise

3. Nouveaux constats à intégrer à l'analyse de risques (WOR-67)

Constat	Risque concerné	Gravité du constat
Port de la base de l'application (55432) publié sur toutes les interfaces, avec un mot de passe en clair dans le dépôt	R5 exfiltration	**Élevée** : la base est joignable depuis le réseau `RESILIENCE-G6`
L'assistant de discussion de l'application envoie les messages des utilisateurs à OpenAI	Fuite de données (ER3), prestataire TIC tiers (DORA art. 28)	Moyenne : données fictives, mais incohérent avec un assistant « local »
Code de vérification des paiements fixe (`123456` par défaut) **et renvoyé dans la réponse de l'API** (`demoCode`)	R4 usurpation, R2 fraude	Moyenne en démo, critique en production : la vérification ne protège rien
Limitation de débit sur la connexion (12 essais / 15 min), mais pas de verrouillage de compte	R4 bourrage d'identifiants	Faible : la limitation freine déjà l'attaque
Vault avec une seule part de clé, clé de descellement et jeton root dans un fichier de l'hôte	R3, R7	Moyenne : acceptable pour un prototype, à citer en limite
Attaque A dirigée sur `/health` du labo IA, attaque B sur `/api/pay`, qui n'existe ni dans l'application ni dans le labo	Validité des essais	**Élevée** : l'attaque B ne modifie aucune donnée, il n'y a donc rien à restaurer. La réponse « restauration » ne peut pas être démontrée en l'état
Pas de solde ni de débit dans le modèle de données (intentions de paiement uniquement)	ER2 altération	Faible : le risque devient « paiements altérés ou dupliqués » ; l'idempotence protège déjà contre les doublons

4. Ce qu'il faut avant les essais de mercredi

Par ordre de priorité, pour que la démo prouve ce que la gouvernance affirme :

1. **Brancher l'API de l'application sur le socle**, derrière le WAF (qui renvoie aujourd'hui 502 faute de backend), collectée par Prometheus et Loki, avec une seule base. Infrastructure, Développement.
2. **Remplacer `/health` par une vérification réelle** : base joignable et transaction de test réussie. Développement.
3. **Construire le moteur de remédiation** (WOR-38), avec la vérification de la reprise. Développement.

4. **Entraîner le modèle et envoyer les alertes** (WOR-52, WOR-53). IA / Data.
5. **Recâbler les attaques A et B** sur les vraies routes de l'API. Pour l'attaque B, **décider ce qu'on démontre** : le blocage par le WAF (déjà obtenu), ou la restauration, qui suppose qu'une injection passe le WAF et écrive en base. Cybersécurité, Gouvernance.
6. **Une sauvegarde de la base avant chaque essai**, au minimum un `pg_dump` horodaté, conservé hors des volumes purgés. Infrastructure.
7. **Fermer le port 55432 et passer les secrets de l'application dans Vault.** Cybersécurité.
8. **Copier les journaux avant toute remise à l'état sain**, pour garder les preuves des essais. Infrastructure.